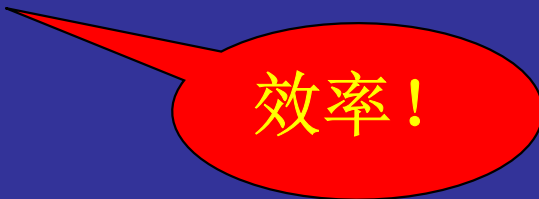


考试成绩：卷面占70%，平时占30%

考试内容：

- 逻辑结构：表（栈、队、串、数组）、树、图
- 存储结构：顺序、链式、散列
- 基本运算：插入、删除、修改、查找、排序



效率！

回顾:

第1章	绪论	
第2章	线性表	线性结构
第3章	栈和队列	
第4章	串	
第5章	数组和广义表	
第6章	树和二叉树	非线性结构
第7章	图	
第9章	查找	高效运算
第10章	排序	

一、线性结构

(包括：表、栈、队、串、数组、广义表)

1. 【01级试题】试用C或类C语言编写一个算法，将一循环单链表就地逆置。（9分）

提示：

“就地”意味着数据不能动；

“逆置”意味着后进先出；

因此用链栈为佳。

算法的核心部分为：

```
q=head;    //保存表尾特征
p=head->next; //假定无头结点
while(p!=q){
    r=p->next;
    p->next=head;
    head=p;
    p=r;}
q->next=head;
```

2. 【2004年北电公司招聘笔试题】 已知两个链表**LA**和**LB**都是有单调递增有序表，请编写算法，将二表合并为一个单调递减的有序表。

提示：

- ① 关于两个有序链表归并为一个有序链表的算法，参见2.4节 应用举例（教材P31例）
- ② 关于“逆置”，参见上题链栈算法。

3. 【03级试题】 设有一个头结点H，H的指针域指向一个双向链表（非循环）的首结点。用C或类C编写一个将此双向链表逆序的算法（请定义H的数据类型）。

解：可以有两种算法：

- ① 先将表中每个结点的两个指针域互换，再将H指向原双向链表的末尾；
- ② 直接寻找到原双向链表的末尾，将H指向表尾；但要重新定义，将原链表的前驱指针现在定义为后继，后继定义为前驱。

注1：两种算法的时间复杂度都是 $O(n)$ ，空间复杂度都是 $O(1)$

注2：头结点H的数据类型与链表结点不必一样，可以只设一个指针分量。

4. 关于时间效率和空间效率的问题

【04级试题】 算法A的时间复杂度为 $O(1)$ ，算法B的时间复杂度是 $O(\log_2 n)$ ，算法A的运算时间一定比算法B少吗？为什么？

答：不一定！ $O(1)$ 表示是常数时间，可能是50，比实际上的 $O(\log_2 n)$ 可能大很多！

【02级试题】 算法复杂度 $O(1)$ 的含义是什么？

答：它表示与输入的元素规模无关，是一个常数（但不一定是1）。

或：它表示该算法执行时耗费时间的长短或占用辅助空间的多少与元素个数 n 无关，若能达到这样的时间效率或空间效率，将是最理想的算法。

栈的常用题：表达式求值、括号配对、递归运算等等。

5. 【04级考题】 设有一个动态分配顺序栈S，元素S1，S2，S3，S4，S5，S6依次进栈，如果6个元素的出栈顺序为S2，S3，S4，S6，S5，S1，则顺序栈的容量至少应为多少？

答：栈的容量至少应为3，（S1+S5+S6）

6. 【04级考题】 设采用多个栈来模拟队列，即只有进栈push和出栈pop操作，请写出进队和出队的算法，并分析算法的时间复杂度。（10分）

答：利用2个栈s1和s2来模拟队列，s1和s2的大小一样为n，则进队只需进栈s1，出队从栈s2中出，当操作栈s2时，需要把栈s1中的所有元素进入栈s2中。（4分）

具体算法如下：

入队(即压入S1)的操作略 (2分)

出队操作如下:

```
status Delcq(Queue s, Elemtyp e &e)
{
    if (empty(s2)) //当栈s2为空, 需要把栈s1中的元素全部
        放入栈s2中, 注意是全部! 当栈s2不空时, 不能把栈s1中的
        元素放入栈s2中。
    { while not empty(s1)
        { pop(s1, e); push(s2, e); }
      } //依次把栈s1中的元素放入栈s2中, 实现序列的反序!
    if (empty(s2)) return(ERROR); //此时栈s2还为空, 表明
    队列是空, 返回队列空标志ERROR
    pop(s2, e); //真正的出队列元素e出s2栈
    return(OK);
}
```

6. 【01级试题】 假设有二维数组 $A_{7 \times 9}$ ，每个元素用相邻的6个字节存储，存储器按字节编址。已知A的起始存储位置（基地址）为1000，末尾元素 $A[6][8]$ 的第一个字节地址为多少？若按列存储时，元素 $A[4][7]$ 的第一个字节地址为多少？

答：

① 末尾元素 $A[6][8]$ 的第一个字节地址 = $1000 + (7 \text{行} \times 9 \text{列} - 1) \times 6B = 1000 + 62 \times 6 = 1372$

② 按列存储时，元素 $A[4][7]$ 的第一个字节地址 = $1000 + (7 \text{列} \times 7 \text{行} + 4) \times 6B = 1000 + 53 \times 6 = 1318$

7. 【03级试题】在KMP算法中，已知模式串为LIUYULIUWENYU，请写出模式串的next[j]函数值，并分析next[j]函数值的大小与KMP算法的比较次数多少（或时间）有何关系，简单解释你的理由。

答：next[j]函数值=0111112341111

next[j]函数值越大，比较次数越多；

反之，函数值越小，则比较次数越少，时间越快。

函数值小，说明模式串各位基本不相关，则向右滑动速度会很快。

8. 【09级试题】在KMP算法中，已知模式串为HKUCUHKHKUSTHUST，请写出模式串的next[j]函数值，设主串和模式的字符个数分别为n和m，给出最坏的情况下KMP算法的时间复杂度。(5分)

9. 【09级试题】编号为1, 2, ..., n的n个人按顺时针方向围坐一圈, 每个人持有一个密码。开始任选一个整数为报数上限值m, 从第一个人开始按顺时针或逆时针方向自1开始报数 (正整数顺时针、负整数逆时针), 至m时停止, 报m的人出列, 将他的密码作为新的m值, 从他的下一个人开始重新从1报数, 直至所有人全部出列为止。设计算法求出出列顺序:

(1) 设计出合适的数据结构;

(2) 根据存储结构编制出类C语言程序。(10分)

有3种方法, 循环队列 (向量)、单循环链表、双向循环链表, 各有优缺点。
注意:

1) m需要取余, 否则m比n大很多会浪费时间, 注意-m取余后是正数, 也即都可以按顺时针方向报数, 这是可以用单循环链表的原因。

2) 圈的长度在变化, 需要记录;

3) 需考虑时间效率, 无论m是什么, 走的路线不能超过半圈, 这是双向循环链表的优点。

二、非线性结构 (包括：二叉树、树、图)

1. 【01级试题】某完全二叉树共有15381个结点，请问其树叶有多少个？

答： $n_2=7691$ (个)

2. 【09级试题】某完全二叉树共有20101207个结点，请问其树叶有多少个？

3. 【04级试题】由1001个结点构成的二叉树，何种形态叶子结点最多，何种叶子最少？分别给出叶子结点个数和度为1结点的个数。

答：完全二叉树叶子最多,叶子结点501个,度为1结点的个数为0
单支树叶子最少,叶子结点个数为1,度为1结点的个数1000

4. 【04级试题】前序遍历的平均空间和最坏空间复杂度是多少？从时间效率上看，前序遍历和后序遍历哪一种更费时间，为什么？

答：

前序遍历的平均空间和最坏空间复杂度是 $O(\log_2 n)$ 和 $O(n)$

后序遍历更费时间，因为后序遍历每个元素需2次进栈或利用标志位进栈！

5. 【02级试题】中序遍历的递归算法平均空间复杂度为多少？

答：要考虑递归时占用了栈空间，但递归次数最多不超过树的高度，所以空间复杂度为 $O(\log_2 n)$

6. 【03级试题】 已知一棵二叉树的前序序列和中序序列分别为： **ABDGCEFH**和**DGBAECFH**，则该二叉树的后序序列是什么？

答：既可先画树而得后序序列，也可直接推出后序序列，结果为：**GDBEHFCA**

7. 【03级试题】 试编写一个完整算法，统计一给定的二叉树中度为1 的结点个数。

提示：用任何一种遍历递归算法，将遍历函数中**Visit()**细化即可。

8. 【01级试题】定义二叉树的宽度为二叉树一层中结点个数的最大值，试编写一算法求二叉树的宽度。（9分）

提示：要用层次遍历以及队列来处理，并一定要设立一个宽度计数器和一个temp，在统计完每一层的结点个数之后就要与计数器中前一层的值比较，保留大值。

可参见严题集【6.52】④

9. 【03级试题】设计一个算法，求二叉树中任一给定结点所在的层次，并分析算法的平均时间和空间复杂度。（设根结点的指针为T，给定结点的指针为x）。

10. 【09级试题】设已经一颗二叉树的根结点为root。请写一个空间复杂度为 $O(1)$ 的算法，输出后序遍历序列的第一个结点的值，并给出该算法的时间复杂度。（10分）

- 1) 若有左儿子，则 $p=p \rightarrow \text{left}$ ，直至 $p \rightarrow \text{left} = \text{NIL}$;
- 2) 若无左儿子，但有右儿子，则 $p=p \rightarrow \text{right}$; goto 1)
- 3) 若既没有左儿子，也没有右儿子，则p即为所求。

11. 通常求树深的算法

```
int BTreeDepth(Btree *BT)  /*BT为二叉树某结点的指针
{ int leftdep, rightdep;    //设左右两个深度/层次计数器
  if(BT==NULL) return(0);  //当前结点指针为空则立即返回
  else
  { leftdep= BTreeDepth(BT->left); //遍历当前结点左子树
    rightdep=BTreeDepth(BT->right); //遍历当前结点右子树
    if( leftdep>rightdep)return(leftdep+1); //从叶子计数
    else return(rightdep+1);
  }
} //BTreeDepth
```

树宽 × 树深 = 树的繁茂度

12. 【02级试题】分析下面算法中l和h变量表示什么含义？初始调用时，l和h应取什么值？其中p为指向二叉树的根结点，如果去掉形参中的“&”符号，会得到什么结果？（10分）

```
Void ABC(Bitree p, int l, int &h)
{ if p≠NIL then
  { l=l+1;
    if l>h then h=l;
    ABC(p->Lchild, l, h);
    ABC(p->Rchild, l, h);
  }
}
```

此题是求树的深度。
但求解方法是从根开始计算层次，反而比一般书籍中从叶子往上计算要简单。

答：依分析，l、h表示二叉树的层次数和深度。

开始调用时，应为ABC (p, 0, 0)

去掉形参中的“&”号，则上次计算的结果不能正确返回。故h不变，得不到正确结果。

13. 【03级试题】 设p是二叉树的根结点，分析下面算法的功能是什么。请问有何错误？若有错，请改为正确的算法，并分析算法的平均时间和空间复杂度。(10分)

```
ABC (bitreptr p)
```

```
{ while p≠NULL
```

```
    { ABC(p->Lchild);
```

```
      x=p-> Lchild; p-> Lchild =p->Rchild; p-> Rchild=x;
```

```
      ABC(p->Rchild);
```

```
    }
```

```
}
```

答：此算法功能是将二叉树左右子树交换。（改错就是交换语句不能插在中间，要么放前面要么放后面）

14. 【09级试题】 有n个结点的二叉树采用顺序结构存储，何种树型最节省存储空间？何种树型最浪费存储空间？ (5分)

答：节省是满二叉树，浪费是单支树

15. 【04级试题】下面是某人设计的判断是否完全二叉树的算法，**p**是根结点，指出下面算法的错误（不需写出正确的算法）

Boolean Iscomplete(BiTree &p)

```
{  if p == NIL return(TRUE);  
    if (p->lchild != NIL && p->rchild ==NIL) return(TRUE);  
    if (p->lchild == NIL && p->rchild !=NIL) return(FALSE)  
    else return (Iscomplete(p->lchild) && Iscomplete(p->rchild));  
}
```

解答：

1. **BiTree &p**中应没有**&**号；
2. **if (p->lchild != NIL && p->rchild ==NIL)**
return(TRUE); 多余，应删除
3. 左右子树都是完全二叉树不能说此二叉树是完全二叉树，此算法的思路有问题，应该用队列按层遍历二叉树的方法
4. **if (p->lchild == NIL && p->rchild !=NIL)**
return(FALSE)后少 “ ; ” 号，不扣分也不得分。

二叉树的常用题有：

1. 先序/中序/后序遍历的非递归算法
2. 层次遍历算法的实现
3. 求二叉树的深度
4. 求二叉树的宽度
5. 如何判定一棵二叉树是完全二叉树？
6. 如何判定一棵二叉树是二叉排序树？

16. 【03级试题】 假设字符a,b,c,d,e,f的使用频度分别是0.07, 0.09, 0.16, 0.18, 0.23, 0.27, 写出a,b,c,d,e,f的Huffman编码并计算其平均码长WPL。

如果对于一个编解码系统, 双方各建一棵Huffman树, 设建树步骤都是首先将各个字符的权重进行排序, 再取最小的两个权值进行合并, 请问是否需要考虑这个排序算法的稳定性, 你选择什么排序方法, 为什么? (12分)

答: 由于是各自建立, 需要稳定算法

17. 【02级试题】 设A~H 8个字符出现的概率为: $\omega = \{0.10, 0.16, 0.01, 0.02, 0.29, 0.10, 0.07, 0.25\}$, 设计最优二进制码并计算平均码长。如果设计最优三进制编码 (即可用0,1,2三种符号进行编码), 画出最优三叉树并计算平均码长。 (10分)

答: 最优二进制码构造同上, WPL=2.59 Huffman树

提示: 对三进制编码, 由于总共有8个字符, $8\%3=2$, 故第一次构建最优树只能合并2个结点。 WPL=1.74 Huffman树

18. 【03级试题】哈夫曼树是带权路径长度之和最小的树，但现在要求带权路径长度之和最大的树，你认为应该怎样求？请写出设计思路。假设字符a, b, c, d, e, f的使用频度分别是0.07, 0.09, 0.16, 0.18, 0.23, 0.27，请画出对应的WPL最大的树并计算WPL的值

解：

- (1) 根据给定的n个权值构成n颗二叉树的集合F
- (2) 在F中选取两颗根结点的权值最大的数作为左右子树构造一棵新的二叉树，且置新的二叉树的根结点的权值为其左、右子树上根结点的权值较小者
- (3) 在F中删除这两棵树，同时将新得到的二叉树加入F中
- (4) 重复 (2) 和 (3)，直到F只含一棵树为止

图

1. 【01级试题】某无向图G的邻接表如下图所示。要求：

- ① 请画出该图G的逻辑结构图；
- ② 根据邻接表写出其深度优先遍历序列和广度优先遍历序列(设访问起点为 v_3)；
- ③ 根据遍历结果，画出图G的深度优先和广度优先生成树。

2. 【02级试题】下图为某无向图的邻接表，按教材算法7.5和7.6分别写出深度优先搜索和广度优先搜索的结果，并画出逻辑结构图。

3. 【04级试题】已知下图的深度优先和广度优先搜索的输出前2个结点①、②，请写出深度优先和广度优先的输出序列。

4. 【09级试题附加题10分】 设一幅地图有 n 个区域（例如，省）。用不多于4种颜色对这些区域进行着色，着色应满足的要求是相邻的区域具有不同的颜色。你的算法以一种着色方案（即哪一个区域着什么颜色）为输入，算法对该着色方案进行考察，若满足着色要求，则输出true,否则输出false。

① 用自然语言和类C语言描述你为解决问题而设计的数据结构（逻辑结构，存储结构）。数据结构的设计应考虑对问题的清楚描述和算法的效率。

② 用类C语言写出你的算法，并分析算法的时间、空间复杂度。

用图结构表示地图，一个顶点表示一个区域，相邻的区域用边相连，则变成对一个图进行顶点着色，相邻的顶点用不同的颜色，不能超过4种颜色。判断是否满足着色要求的算法为：

用DFS或WFS对图进行遍历，在遍历的过程中判此顶点与所有相邻的顶点颜色是否相同，如果是一种新的颜色，颜色计数器加1；

如果顶点与所有相邻的顶点颜色相同或颜色计数器 >4 ，**return false**，否则，**return true**；

三、查找和排序

查找算法包括：顺序、折半、二叉排序树、散列
排序算法包括：插入、交换、选择、基数

1. 【01级试题】 假设一有序表中有23个元素，现进行折半查找，则平均查找长度是多少？

提示：不能简单的用 $\log_2 n$ 来计算。因为这是在假设 $n=2^m-1$ 的情况下推导出来的公式，当 n 很大时才能忽略该前提。现在 $n=23$ 比较小，应当用穷举法罗列：

全部元素的查找次数为 $= (1 + 2 \times 2 + 4 \times 3 + 8 \times 4 + 8 \times 5) = 89$

$ASL = 89 / 23 = 3.87$

2. 【01级试题】已知一组关键字为（10，24，32，17，31，30，46，47，40，63，49），设哈希函数 $H(key) = key \text{ MOD } 7$ 。请解答：

- (1) 写出用链地址法处理冲突构造所得的哈希表；
- (2) 若查找关键字17，需要依次与哪些关键字进行比较？
- (3) 若查找关键字60，需要依次与哪些关键字比较？
- (4) 假定每个关键字的查找概率相等，求查找成功时的平均查找长度。 (7分)

答： $ASL \approx 1.82$

3. 【02级试题】已知一组关键字为（10，24，32，17，31，30，46，47，40，63，49），设哈希函数 $H(key) = key \text{ MOD } 13$ 。请写出用线性探测法处理冲突构造所得的哈希表。 (4分)

4. 【09级试题】 已知一个线性表（38， 25， 74， 63， 52， 48）， 设哈希函数 $H(\text{key}) = \text{key} \bmod 7$ ， 计算散列地址， 并散列存储在散列表A[0...6]中， 若处理冲突的方法采用 $H_i = (H(\text{key}) + i) \% 7$ ， $i = 1, \dots, 6$ 。 请画出哈希表， 并回答在该散列表上进行等概率成功查找的平均长度为多少？ （7分）

5. 【09级试题】对 n 个元素的无序序列进行堆排序时，为什么只需从 $n/2$ 处开始建堆？判断一个序列是否是一个堆（**heap**）的算法时间复杂度是多少？（5分）

6. 【09级试题】假设一有序表中有13个元素，现进行折半查找，则平均查找长度是多少？（5分）

7. 【09级试题】 n 个元素的序列，若采用锦标赛排序方法，排出第一个元素的比较次数是多少？排出第10个元素的次数是多少（5分）

8. 【09级试题】如果一个递归算法的时间函数为 $T(n)=3n+2T(n/2)$ ，该算法的时间复杂度是多少？（5分）

9. 【09级试题】已知序列(23,45,46,57,78,89,93)的查找概率分别为(0.2,0.01,0.09,0.1,0.18,0.12,0.3)，按照次优方法进行查找，请构造出次优查找树。（5分）

10. 【04级试题】 n 个整数的序列，求前3个最大值，最少需要多少次比较，为什么？（8分）

答：利用锦标赛算法,共进行 $n-1$ 次比较可得树根是最大值；
第二最大值从直接和最大值元素进行比较过的元素中选，共有 $\log_2 n$ 个（完全二叉树的深度），故需进行 $\log_2 n - 1$ 次比较。
以此类推

11. 【01级试题】欲将无序序列（23, 78, 12, 35, 69, 95, 11, 09, 35*, 48, 99, 26）中的关键码按升序重新排列，请写出快速排序第一趟排序的结果序列。另外请画出堆排序（小根堆）的初始堆。

提示1：快速排序注意要按振荡式逼近算法来实现；

第一趟排序的结果为：

09, 11, 12, [23], 69, 95, 35, 78, 35*, 48, 99, 26

提示2：堆排序要从排无序堆开始，从最后一个非终端结点开始，自下而上调整，而且要排成小根堆。

初始堆序列为：

09, 23, 11, 35, 48, 26, 12, 78, 35*, 69, 99, 95

12. 【04级试题】：采用一个辅助空间的堆排序算法对关键字序列{46, 37, 65, 96, 75, 11, 25, 48}排序，如果排序结果为逆序，请给出初始堆。

答：最终逆序则意味着要建小根堆：11 37 25 48 75 65 46
96

13. 【02级试题】试编写一个算法，判断一给定的整型数组 $a[n]$ 是不是一个堆。

提示：堆的特点是： $k_i \leq k_{2i} \& k_{2i+1}$ 或 $k_i \geq k_{2i} \& k_{2i+1}$

算法如下：

```
void SortA(sqlist &A, int n)
{ if(n==0) return(0); //空表
if (a[1]<a[2]) //由头两个元素判断是大根堆或小根堆
{ for( i=1; i<=n/2; i++) if (a[i]>a[2*i]|| a[i]>a[2*i+1])return(-1);
return(minleap) };
else
{ for( i=1; i<=n/2; i++) if (a[i]<a[2*i]|| a[i]<a[2*i+1])return(-1);
return(“maxleap”)
};
}
```

14. 【04级试题】快速排序在什么情况下最快？什么样情况下最慢？在最慢的情况下请提出一种提高效率的解决办法。

答：快速排序在无序状态下最快，在基本有序（或逆序）下最慢。

提高效率的解决办法为，选中间位置的元素 a_{mid} 为中枢

15. 【02级试题】设有1000个无序的元素，需排出前10个最大（小）的元素，你认为采用哪种排序方法最快？为什么？

答：锦标赛排序或堆排序

16 【04级试题】： 一个整数数组，长度为n，编写一个最优算法实现数组的奇数排在前，偶数排在后。并分析该算法的平均时间和空间复杂度。（10分）

答：一个简单的算法是：

```
int sort(int &a, int low, int high)
{   while(low<high)
    {
        while((low<high)&&a[low]%2) low++;
        while((low<high)&&!a[high]%2) high--;
        if(low<high)
        {
            temp=a[low];
            a[low]=a[high];
            a[high] = temp ;
            low++;
            high--
        }
    }
    return low;
} //sort
```